We describe the attributes of the student using name-value pairs. Name fields are always strings in quotes, and values conform to JSON's data types (`int`, `string`, `boolean`, and so on). The inner objects can be represented as nested objects, which JSON supports, such as the `address` field for the `student` document. Note that JSON doesn't have a date type defined, but industry practice is to provide the date and time data as a string (preferably in ISO 8601 format: `yyyy-MM-dd`, or with a time component, `yyyy-MM-ddThh:mm:ss`). Elasticsearch parses stringified date information to extract data based on the schema definition.

Elasticsearch uses the JSON parser to unmarshal data into appropriate types based on mapping rules available on the index. Each index has a set of mapping rules that Elasticsearch applies when a document is being indexed or a search query is executed. We learn more about mapping in chapter 4.

Elasticsearch reads values from the JSON document and converts them into specific data types for analysis and storing. Just as JSON supports data nesting (a top-level object consisting of a next-level object wrapped up again with another object underneath), Elasticsearch also supports nested data structures.

RELATIONAL DATABASE ANALOGY

If you have some understanding of relational databases, the analogy shown in figure 3.4 may help. Remember, Elasticsearch can be used as a storage server (although I wouldn't suggest using a primary storage service—we discuss the reasons later) just like any database. Comparing and contrasting Elasticsearch against a database is fair. The JSON document from figure 3.3 is equivalent to a record in the table of a relational database (figure 3.4).

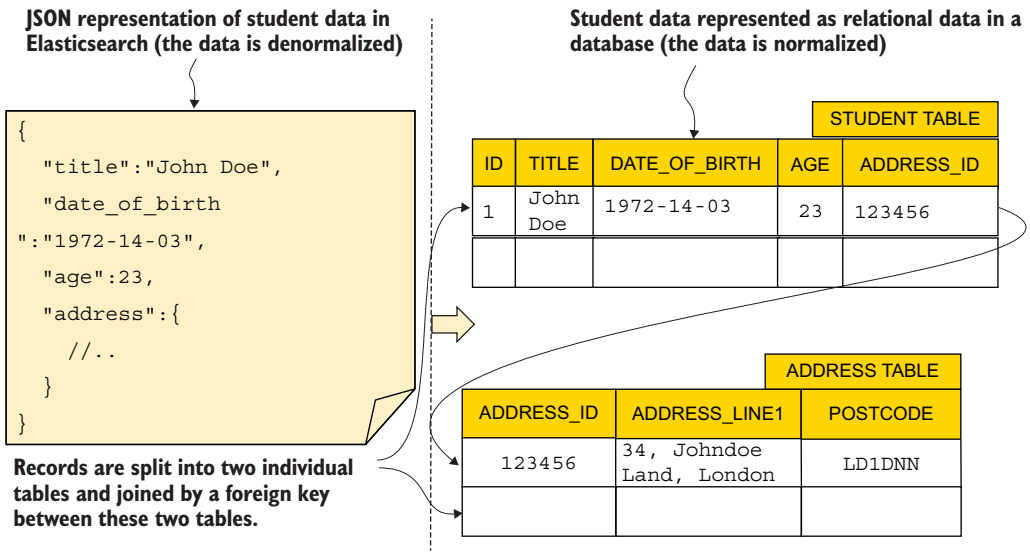


Figure 3.4 A JSON document vs. a relational database table structure